

# Level 1. What a system is

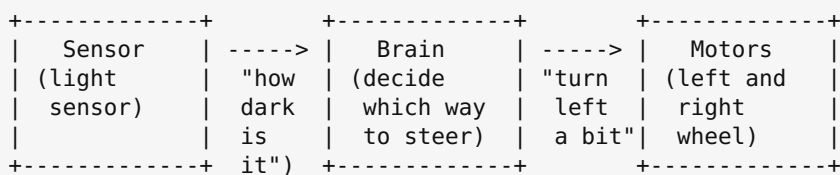
This level has no code. Its job is to give you the four words everything else is built on: **component**, **interface**, **data flow**, and **trade-off**. If you can draw a robot as boxes and arrows and say what travels along each arrow, you have finished level 1.

Rough time: one evening.

## 1.1 A system is boxes and arrows

A system is a set of parts that work together to do one job. To design a system you draw the parts as boxes and the things that pass between them as arrows.

Here is the simplest possible robot system, a robot that follows a black line on the floor:



Three boxes, two arrows. That is already a design. Every robot you will ever work on, from this one to a self-driving car, is this same picture with more boxes:

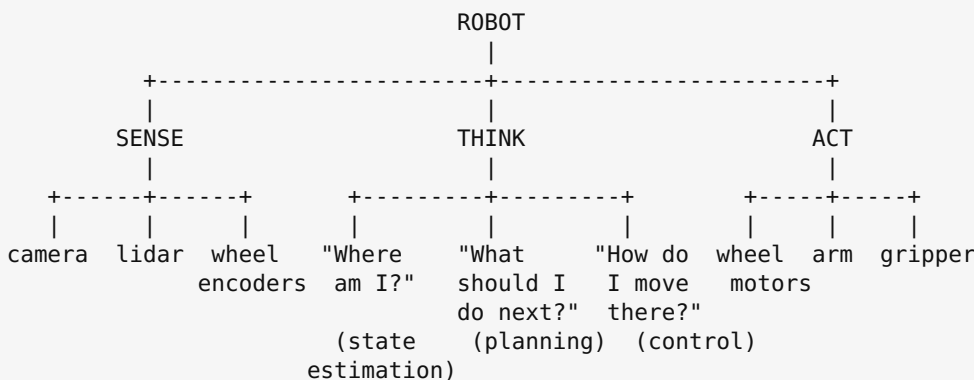
```

SENSE -----> THINK -----> ACT

```

Sense: cameras, lidar, wheel encoders, GPS, microphones. Think: everything that turns sensor data into a decision. Act: motors, arms, grippers, lights, speakers. Keep this triangle in your head. When someone asks you to "design a robot that does X", start by drawing it.

Each of the three words is really a family of boxes. Drawn as a tree, the same robot looks like this:



Read it top down: the robot is made of three families, and each family is made of a few boxes. Sense and Act are usually short lists of hardware. Think is where the software lives, and it always splits into at least three questions: where am I (state estimation), what should I do next (planning), and how do I move to do it (control). Level 2 opens the Think branch fully; for now, just notice that the straight line and the tree are the same picture, and the tree is the one you should draw when someone asks "what is inside the brain?".

## 1.2 The four words

**Component.** A box. It does one job and you can describe that job in a sentence. "Reads the camera and produces images." "Takes a goal and produces a path." A good component can be replaced by a different one that does the same job without the rest of the system noticing. Real example: swapping one lidar for another should only change the lidar driver box.

**Interface.** The shape of an arrow. It says *what* travels between two boxes and *how often*. "An image, 640 by 480 pixels, 30 times a second." "A path, as a list of (x, y) points, whenever the goal changes." Writing down interfaces is most of system design; the boxes are the easy part. If two people agree on the interface, they can build the two boxes separately.

**Data flow.** The direction of the arrows, and the order things happen in. Sensor data flows forward towards the motors. Commands flow forward too. Some arrows flow backwards: "the motor reports its actual speed back to the brain". Drawing the flow shows you where the delays are and where a failure will spread.

**Trade-off.** Every design decision makes one thing better and another thing worse. A faster camera gives fresher pictures but more data to process. A bigger battery lasts longer but makes the robot heavier and slower. There is no design without trade-offs, and saying which trade-off you chose and why is the whole skill. "I chose X because Y matters more than Z here" is the sentence you will say over and over.

## 1.3 A worked example: a robot vacuum cleaner

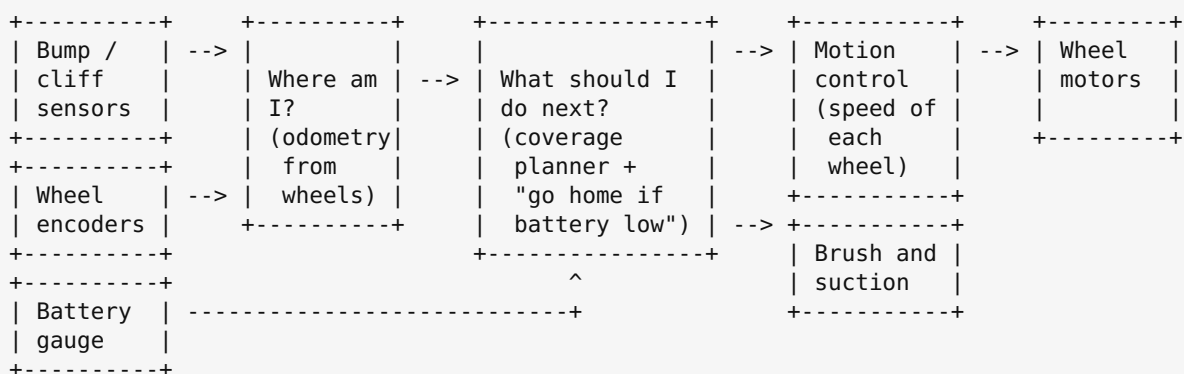
Let us design one with just these four words. The job: clean the floor of a flat without falling down the stairs or getting stuck.

**Step 1. What does it sense?** Bump sensors (did I hit something), cliff sensors (is there floor under me), wheel encoders (how far have I rolled), a battery gauge. Maybe a small camera or lidar on the expensive models.

**Step 2. What does it do?** Drive two wheels, run the brush and suction, and go home to charge.

**Step 3. What must it think about?** Where am I, where have I been, where have I not cleaned yet, am I stuck, is the battery low.

**Step 4. Draw the boxes.**



**Step 5. Write the interfaces.** Pick two arrows and say what travels along them:

| Arrow                             | What travels                                   | How often          |
|-----------------------------------|------------------------------------------------|--------------------|
| Wheel encoders → Where am I       | two numbers: ticks of the left and right wheel | 100 times a second |
| Where am I → What should I do     | the robot's (x, y, heading) estimate           | 20 times a second  |
| What should I do → Motion control | a target speed and turn rate                   | 20 times a second  |

| Arrow                           | What travels                            | How often                   |
|---------------------------------|-----------------------------------------|-----------------------------|
| Bump sensors → What should I do | "hit something on the left/right/front" | immediately when it happens |

**Step 6. Name a trade-off.** Wheel encoders drift: after ten minutes the robot's idea of where it is can be off by a metre. A lidar fixes that but costs ten times more and needs a faster brain. Cheap models accept the drift and clean in random bounces; expensive models add the lidar and clean in neat rows. Same job, different trade-off, both are valid designs.

That is a complete level-1 system design. Notice that you never wrote code, never named a programming language, and never said "ROS". You drew boxes, named what flows between them, and said one thing you chose and why.

## 1.4 Three questions to ask about any arrow

Once the boxes are drawn, the interesting problems all live on the arrows. For each arrow ask:

1. **How fresh is it?** The time between something happening in the world and the box at the end of the arrow knowing about it is called **latency**. A bump sensor with half a second of latency means the robot keeps driving into the wall for half a second. A control loop usually needs latency under a few tens of milliseconds.
2. **How much of it is there?** The amount of data per second is called **throughput** (or bandwidth). A camera at 30 frames a second of 640 by 480 colour pixels is about 28 megabytes a second. Two cameras plus a lidar can saturate a small computer's memory bus. Encoders are a few bytes each.
3. **What happens if it stops?** If the box at the start of the arrow dies or the cable falls out, what does the box at the end do? Keep using the last value? Stop the robot? Switch to a backup? Deciding this for every arrow is what makes a robot safe. The vacuum should stop if the cliff sensor goes silent, because "no reading" might mean "about to fall down stairs".

These three questions, latency, throughput and failure, are the questions a robotics system design conversation is actually about. Level 3 goes deep on them.

## 1.5 How to draw a diagram that people can read

- Left to right, sense on the left, act on the right.
- One job per box, and write the job as a verb phrase inside it.
- Label every arrow with *what* travels on it. An unlabelled arrow is a question you have not answered.
- Put timing on arrows where it matters ("30 Hz", "on change", "once at startup").
- Draw failure arrows in a different way, for example dashed, or list them below the diagram.
- Six to ten boxes fits on a whiteboard. If you need more, group boxes into a bigger box ("Perception") and draw the inside of that box separately.

## 1.6 Try it

Draw, on paper, the boxes and arrows for these three robots. Label every arrow. Name one trade-off for each.

1. A robot arm that picks items from a bin and puts them in a box, using one camera above the bin.
2. A delivery robot that drives on pavements from a shop to a house.

3. A drone that inspects a wind turbine blade for cracks.

Then compare with the answers in level 5, which works these three examples end to end.

---

## 1.7 In your own words

---

A system is components joined by interfaces; the interfaces say what data flows and how often. Every robot is sense, think, act. Designing means drawing the boxes, labelling the arrows, and choosing trade-offs on purpose. The hard questions live on the arrows: latency, throughput, and what happens when an arrow stops.